Operating System Practicals - Official Lab Manual & Viva Guide

1. Shell Programming (Address Book)

File Name: `address_book.sh` **Description:** Interactive shell script providing options to create, view, insert, search, and delete records stored in a plain text database.

Shell

```
#!/bin/bash
filename="address_book.txt"

while true; do
    echo -e "\n--- Address Book Menu ---"
    echo "1. Create Address Book"
    echo "2. View Address Book"
    echo "3. Insert Record"
    echo "4. Search Record"
    echo "5. Delete Record"
    echo "6. Exit"
    read -p "Enter your choice: " choice

    case $choice in
        1)
            > "$filename"
            echo "Address book created successfully."
            ;;
        2)
            if [ -f "$filename" ]; then
                echo "--- Records ---"
                cat "$filename"
            else
                echo "Address book does not exist."
            fi
            ;;
        3)
            read -p "Enter Name: " name
            read -p "Enter Phone: " phone
            read -p "Enter Email: " email
            echo "$name | $phone | $email" >> "$filename"
            echo "Record added."
            ;;
        4)
            read -p "Enter name to search: " sname
```

```
            grep -i "$sname" "$filename" || echo "Record not found."
            ;;
        5)
            read -p "Enter name to delete: " dname
            grep -iv "$dname" "$filename" > temp.txt && mv temp.txt "$filename"
            echo "Record deleted if it existed."
            ;;
        6)
            exit 0
            ;;
        *)
            echo "Invalid choice!"
            ;;
    esac
done
```

Execution:

```
Shell
chmod +x address_book.sh
./address_book.sh
```

Viva Preparation & Notes:

- **Shebang (#!/bin/bash):** Tells the kernel which interpreter to use to execute the script commands.
- **grep options:** `-i` performs case-insensitive search; `-v` selects non-matching lines (used for record deletion).

---

2. Process Management (Fork & Sorting Array)

File Name: `fork_sort.c` **Description:** Demonstrates process creation using `fork()`. Parent and child processes sort the same array of integers using different sorting techniques (e.g., Quick Sort and Merge Sort).

```
C/C++
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
```

```c
#include <sys/wait.h>

void bubbleSort(int a[], int n) {
    for (int i = 0; i < n - 1; i++)
        for (int j = 0; j < n - i - 1; j++)
            if (a[j] > a[j + 1]) {
                int temp = a[j]; a[j] = a[j + 1]; a[j + 1] = temp;
            }
}

void insertionSort(int a[], int n) {
    for (int i = 1; i < n; i++) {
        int key = a[i], j = i - 1;
        while (j >= 0 && a[j] > key) {
            a[j + 1] = a[j];
            j--;
        }
        a[j + 1] = key;
    }
}

int main() {
    int n;
    printf("Enter size of array: ");
    scanf("%d", &n);
    int arr[n];
    printf("Enter elements: ");
    for (int i = 0; i < n; i++) scanf("%d", &arr[i]);

    pid_t pid = fork();
    if (pid < 0) {
        perror("Fork failed");
        return 1;
    } else if (pid == 0) {
        printf("\n[Child Process PID: %d] Sorting using Insertion Sort...\n",
getpid());
        insertionSort(arr, n);
        printf("[Child Process] Sorted Array: ");
        for (int i = 0; i < n; i++) printf("%d ", arr[i]);
        printf("\n");
    } else {
        wait(NULL);
        printf("\n[Parent Process PID: %d] Sorting using Bubble Sort...\n",
getpid());
        bubbleSort(arr, n);
        printf("[Parent Process] Sorted Array: ");
        for (int i = 0; i < n; i++) printf("%d ", arr[i]);
        printf("\n");
```

```
    }
    return 0;
}
```

Execution:

```
Shell
gcc fork_sort.c -o fork_sort
./fork_sort
```

Viva Preparation & Notes:

- **fork():** System call used to create a child process. Returns 0 to the child, child PID to the parent, and -1 on error.
- **wait():** Suspends parent execution until a child process terminates.

---

3. Process States (Zombie and Orphan Processes)

File Name: `zombie_orphan.c` **Description:** Demonstrates the creation and prevention of Zombie and Orphan process states in C.

```
C/C++
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();
    if (pid < 0) {
        perror("Fork failed");
        return 1;
    } else if (pid == 0) {
        printf("[Child] PID: %d, Parent PID: %d\n", getpid(), getppid());
        sleep(5); // Sleep to demonstrate orphan state if parent dies first
        printf("[Child] Adopted Parent PID: %d\n", getppid());
    } else {
        printf("[Parent] PID: %d. Terminating to leave child as orphan.\n",
getpid());
```

```
        exit(0);
    }
    return 0;
}
```

Execution:

```
Shell
gcc zombie_orphan.c -o zombie_orphan
./zombie_orphan
```

Viva Preparation & Notes:

- **Zombie Process:** A process that has completed execution but still has an entry in the process table because its parent hasn't read its exit status with `wait()`.
- **Orphan Process:** A running process whose parent process has terminated. It is adopted by `init` (PID 1) or systemd.

---

4. Thread Synchronization (Producer-Consumer using Semaphores & Mutex)

File Name: `prod_cons.c` **Description:** Solves the producer-consumer problem using POSIX threads, counting semaphores (`empty`, `full`), and a mutual exclusion mutex.

```
C/C++
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 5

int buffer[BUFFER_SIZE];
int in = 0, out = 0;
sem_t mutex, full, empty;

void* producer(void* arg) {
    for (int i = 0; i < 5; i++) {
```

```
        int item = rand() % 100;
        sem_wait(&empty);
        sem_wait(&mutex);

        buffer[in] = item;
        printf("Produced: %d at index %d\n", item, in);
        in = (in + 1) % BUFFER_SIZE;

        sem_post(&mutex);
        sem_post(&full);
        sleep(1);
    }
    pthread_exit(NULL);
}

void* consumer(void* arg) {
    for (int i = 0; i < 5; i++) {
        sem_wait(&full);
        sem_wait(&mutex);

        int item = buffer[out];
        printf("Consumed: %d from index %d\n", item, out);
        out = (out + 1) % BUFFER_SIZE;

        sem_post(&mutex);
        sem_post(&empty);
        sleep(2);
    }
    pthread_exit(NULL);
}

int main() {
    pthread_t prod, cons;
    sem_init(&mutex, 0, 1);
    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);

    pthread_create(&prod, NULL, producer, NULL);
    pthread_create(&cons, NULL, consumer, NULL);

    pthread_join(prod, NULL);
    pthread_join(cons, NULL);

    sem_destroy(&mutex);
    sem_destroy(&full);
    sem_destroy(&empty);
    return 0;
}
```

Execution:

```
Shell
gcc prod_cons.c -lpthread -lrt -o prod_cons
./prod_cons
```

Viva Preparation & Notes:

- **Mutex vs. Semaphore:** A mutex is a locking mechanism (binary) owned by a single thread, whereas a counting semaphore tracks resource availability across multiple units.
- **sem_wait & sem_post:** `sem_wait()` decrements the semaphore (blocks if 0); `sem_post()` increments it.

---

5. Thread Synchronization (Dining Philosophers Problem)

File Name: `dining_philosophers.c` **Description:** Solves the classic Dining Philosophers synchronization problem using semaphores to prevent deadlock.

```
C/C++
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define N 5
#define THINKING 2
#define HUNGRY 1
#define EATING 0
#define LEFT (phnum + 4) % N
#define RIGHT (phnum + 1) % N

int state[N];
int phil[N] = { 0, 1, 2, 3, 4 };
sem_t mutex, S[N];

void test(int phnum) {
    if (state[phnum] == HUNGRY && state[LEFT] != EATING && state[RIGHT] != EATING)
{
        state[phnum] = EATING;
        sleep(1);
```

```
        printf("Philosopher %d takes fork %d and %d and Eats\n", phnum + 1, LEFT +
1, phnum + 1);
        sem_post(&S[phnum]);
    }
}

void take_fork(int phnum) {
    sem_wait(&mutex);
    state[phnum] = HUNGRY;
    test(phnum);
    sem_post(&mutex);
    sem_wait(&S[phnum]);
}

void put_fork(int phnum) {
    sem_wait(&mutex);
    state[phnum] = THINKING;
    printf("Philosopher %d putting fork %d and %d down\n", phnum + 1, LEFT + 1,
phnum + 1);
    test(LEFT);
    test(RIGHT);
    sem_post(&mutex);
}

void* philosopher(void* num) {
    int* i = num;
    take_fork(*i);
    put_fork(*i);
    return NULL;
}

int main() {
    pthread_t thread_id[N];
    sem_init(&mutex, 0, 1);
    for (int i = 0; i < N; i++) sem_init(&S[i], 0, 0);
    for (int i = 0; i < N; i++) pthread_create(&thread_id[i], NULL, philosopher,
&phil[i]);
    for (int i = 0; i < N; i++) pthread_join(thread_id[i], NULL);
    return 0;
}
```

Execution:

```
Shell
gcc dining_philosophers.c -lpthread -o dining_philosophers
```

```
./dining_philosophers
```

Viva Preparation & Notes:

- **Deadlock Condition:** Deadlock occurs if all philosophers grab their left fork simultaneously and wait indefinitely for the right fork.

---

6. Deadlock Avoidance (Banker's Algorithm)

File Name: `bankers.c` **Description:** Determines if a multi-process resource allocation system is in a safe state and prints the safe execution sequence.

```C/C++
#include <stdio.h>

int main() {
    int n = 5, m = 3;
    int alloc[5][3] = { {0, 1, 0}, {2, 0, 0}, {3, 0, 2}, {2, 1, 1}, {0, 0, 2} };
    int max[5][3]   = { {7, 5, 3}, {3, 2, 2}, {9, 0, 2}, {2, 2, 2}, {4, 3, 3} };
    int avail[3]    = { 3, 3, 2 };

    int f[5], ans[5], ind = 0;
    for (int k = 0; k < n; k++) f[k] = 0;

    int need[5][3];
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            need[i][j] = max[i][j] - alloc[i][j];

    for (int k = 0; k < 5; k++) {
        for (int i = 0; i < n; i++) {
            if (f[i] == 0) {
                int flag = 0;
                for (int j = 0; j < m; j++) {
                    if (need[i][j] > avail[j]) {
                        flag = 1;
                        break;
                    }
                }
                if (flag == 0) {
                    ans[ind++] = i;
                    for (int y = 0; y < m; y++)
                        avail[y] += alloc[i][y];
```

```
                    f[i] = 1;
                }
            }
        }
    }

    int safe = 1;
    for (int i = 0; i < n; i++) {
        if (f[i] == 0) { safe = 0; break; }
    }

    if (safe == 1) {
        printf("System is SAFE. Safe Sequence:\n");
        for (int i = 0; i < n - 1; i++) printf("P%d -> ", ans[i]);
        printf("P%d\n", ans[n - 1]);
    } else {
        printf("System is UNSAFE!\n");
    }
    return 0;
}
```

Execution:

```
Shell
gcc bankers.c -o bankers
./bankers
```

Viva Preparation & Notes:

- **Need Matrix:** Calculated as `Need[i][j] = Max[i][j] - Allocation[i][j]`.
- **Safe State:** A state where there exists at least one sequence of execution that allows all processes to finish without deadlock.

---

7. Interprocess Communication (Anonymous Pipes)

File Name: `pipes_comm.c` **Description:** Parent passes a file path to child via Pipe 1, and child responds back with confirmation/content via Pipe 2.

```
C/C++
#include <stdio.h>
```

```c
#include <unistd.h>
#include <string.h>

int main() {
    int pipe1[2], pipe2[2];
    pid_t pid;
    pipe(pipe1);
    pipe(pipe2);

    pid = fork();
    if (pid == 0) {
        close(pipe1[1]);
        close(pipe2[0]);

        char path[50];
        read(pipe1[0], path, sizeof(path));
        printf("[Child] Received path: %s\n", path);

        char *msg = "File contents loaded and processed successfully.";
        write(pipe2[1], msg, strlen(msg) + 1);
    } else {
        close(pipe1[0]);
        close(pipe2[1]);

        char *path = "sample.txt";
        write(pipe1[1], path, strlen(path) + 1);

        char buffer[150];
        read(pipe2[0], buffer, sizeof(buffer));
        printf("[Parent] Received from child:\n%s\n", buffer);
    }
    return 0;
}
```

Execution:

```shell
gcc pipes_comm.c -o pipes_comm
./pipes_comm
```

Viva Preparation & Notes:

- **Pipe Characteristics:** Unidirectional communication channel. Requiring two pipes for full-duplex communication between parent and child.

---

8. Interprocess Communication (Named Pipes / FIFOs)

File Name: `fifo_writer.c & fifo_reader.c` **Description:** Demonstrates IPC between two unrelated processes using named pipes created with `mkfifo()`.

```C/C++
// Writer Process: fifo_writer.c
#include <stdio.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <unistd.h>

int main() {
    char *fifo_path = "/tmp/myfifo";
    mkfifo(fifo_path, 0666);
    printf("Writer: Opening FIFO...\n");
    int fd = open(fifo_path, O_WRONLY);
    write(fd, "Hello from FIFO Writer!", 24);
    close(fd);
    return 0;
}
```

Execution:

```Shell
gcc fifo_writer.c -o fifo_writer
./fifo_writer
```

Viva Preparation & Notes:

- **FIFO vs Anonymous Pipe:** FIFOs exist as special files in the filesystem and enable IPC between unrelated processes, whereas standard pipes require a parent-child hierarchy.

---

9. Disk Scheduling Algorithms (SSTF, SCAN, C-LOOK)

File Name: `disk_sched.c` **Description:** Implementation of Shortest Seek Time First (SSTF), SCAN (Elevator), and C-LOOK disk scheduling algorithms.

C/C++

```c
#include <stdio.h>
#include <stdlib.h>

void sstf(int requests[], int n, int head) {
    int total_seek = 0;
    int visited[10] = {0};

    printf("Starting Head Position: %d\nPath: %d", head, head);
    for (int i = 0; i < n; i++) {
        int min = 999, idx = -1;
        for (int j = 0; j < n; j++) {
            if (!visited[j]) {
                int dist = abs(requests[j] - head);
                if (dist < min) {
                    min = dist;
                    idx = j;
                }
            }
        }
        visited[idx] = 1;
        total_seek += min;
        head = requests[idx];
        printf(" -> %d", head);
    }
    printf("\nTotal Seek Time: %d tracks\n", total_seek);
}

int main() {
    int n = 7, head = 50;
    int requests[] = {82, 170, 43, 140, 24, 16, 190};
    sstf(requests, n, head);
    return 0;
}
```

Execution:

Shell

```
gcc disk_sched.c -o disk_sched
./disk_sched
```

Viva Preparation & Notes:

- **SSTF:** Selects the request with the minimum seek time from the current head position (can cause starvation).

- **SCAN:** Moves head toward one end servicing requests until reaching the end, then reverses direction.
- **C-LOOK:** Goes as far as the last request in one direction, then immediately jumps back to the furthest request in the opposite direction without going all the way to the disk edge.